

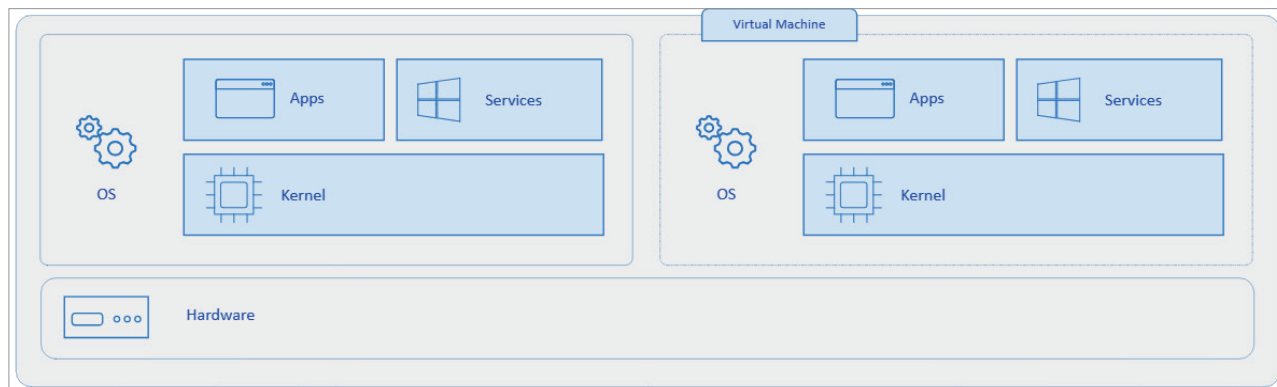


Figure 2: A virtual machine

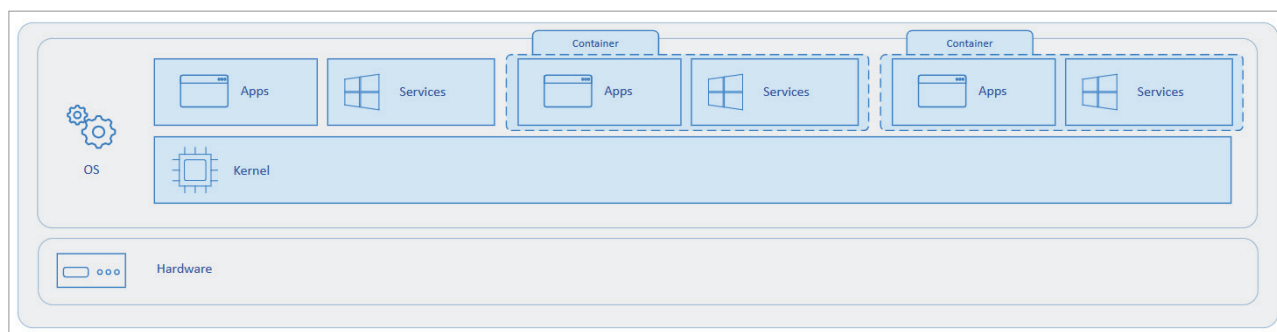


Figure 3: Containers

Benefits of VMs	Limitations of VMs
Run different operating systems on the same hardware.	Each VM includes a full OS, consuming more CPU and memory.
Run older software not compatible with modern systems.	VMs take more time to start due to full OS load.
Create isolated and resettable testing environments.	Scaling many VMs requires advanced tools and infrastructure.
Easy to back up and restore entire VMs quickly.	Large disk images increase storage needs and costs.

Table 1: Benefits and limitations of VMs

What is a container?

A container is a lightweight, standalone unit that packages an application along with all its required libraries, configurations, and dependencies. Unlike virtual machines, containers share the host operating system kernel, which makes them faster and more efficient. They create a consistent environment for running applications across different platforms.

How containers work: Containers run on top of a

container engine such as Docker, containerd, or CRI-O. Instead of virtualising hardware, they leverage the host's OS and isolate processes using Linux features like namespaces (for separation) and control groups (cgroups) for resource management.

This allows multiple containers to run on the same machine without the overhead of separate operating systems. Containers can start and stop almost instantly, making them ideal for tasks that require quick deployment or scaling. Their benefits and limitations are listed in Table 2.

Before diving into deployment, it's important to understand how containers and virtual machines differ at a fundamental level. Table 3 highlights the key differences to help you choose the right solution for your environment.

How to choose between containers and VMs

Choosing between containers and virtual machines depends on your specific application needs, infrastructure setup, and scalability goals. Now that you've seen the core differences, let's explore when each option makes the most sense in practical scenarios.

Containers are primarily valuable when your aim is to build cloud-native modern applications that leverage cloud-native platforms in ways that emphasise speed, consistency, and scalability, such as microservices environments, CI/CD pipelines, or cloud environments. Developers enjoy